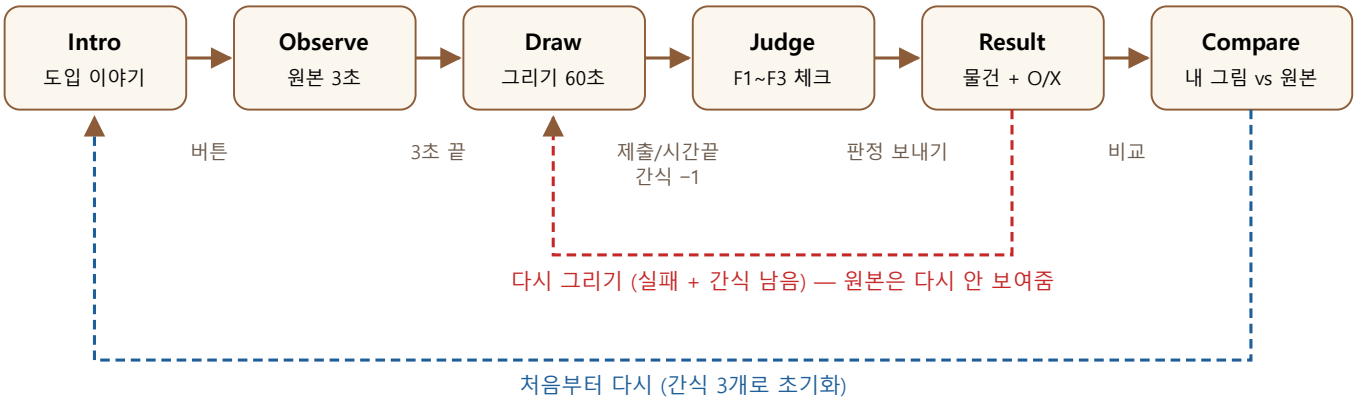
# 찾아라! 도깨비 — MVP 아키텍처

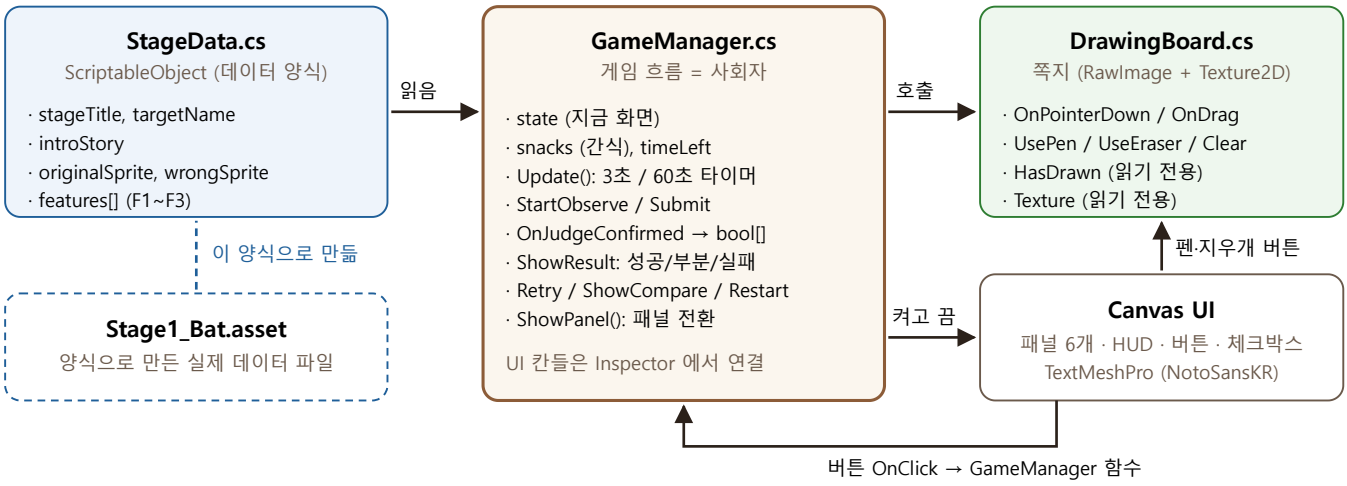
1단계 방망이 · Unity 6.3 LTS · 씬 1개 + 패널 6개 + 스크립트 3개 · 판정은 테스트 입력(체크박스)

## 1. 플레이 흐름 (GameManager 의 State)




> 화면을 바꾸는 방법은 하나뿐입니다: `ShowPanel(패널)` — 그 패널만 켜고 나머지 5개는 끕니다.

## 2. 구성 요소와 의존 관계




> 화살표 방향 = "누가 누구를 아는가". **DrawingBoard 와 StageData 는 GameManager 를 모릅니다.** 그래서 각각 따로 고치거나 재사용하기 쉽습니다.

## 3. 데이터 흐름 — 그림 한 장이 결과가 되기까지

| 단계 | 어디서 → 어디로 | 무엇이 전달되나 |
|---|---|---|
| 게임 시작 | Stage1_Bat.asset → GameManager | `stage` 칸에 연결된 데이터 전체 (이름, 이야기, 그림 2장, 특징 3개) |
| 관찰 | GameManager → TargetImage | `stage.originalSprite` 를 3초 보여줌 |
| 그리기 | 마우스 → DrawingBoard | 화면 좌표 → 픽셀 좌표 → `pixels[]` 배열 색 변경 → `Texture` |
| 그리기 중 | DrawingBoard → GameManager | `HasDrawn` → 제출 버튼 `interactable` |
| 제출 | GameManager 내부 | `snacks--`, 체크박스 초기화, 판정 화면으로 |
| 판정 | 체크박스 ×3 → GameManager | `bool[] present` 예) {true, true, false} |
| 결과 | GameManager → ResultPanel | true 개수 → 제목·가져온 그림·[O]/[X] 목록·버튼 표시 여부 |
| 비교 | DrawingBoard → MyDrawing<br>StageData → Original | 내가 그린 `Texture` 를 그대로 재사용 / 원본 Sprite |

## 4. 결과 계산 규칙 (ShowResult)

| 맞힌 특징 수 | 결과 | 도깨비가 가져오는 것 | 버튼 |
|---|---|---|---|
| 3 / 3 | **성공** | 진짜 방망이 (originalSprite) | 내 그림과 비교하기 |
| 1 ~ 2 / 3 | **부분 성공** | 엉뚱한 방망이 (wrongSprite) | 다시 그리기 (간식 남으면) · 포기하고 비교 |
| 0 / 3 | **실패** | 아무것도 없음 | 다시 그리기 (간식 남으면) · 포기하고 비교 |

> 간식은 3개로 시작, 제출할 때마다 1개. 0개면 다시 그리기 버튼이 사라지고 "간식이 다 떨어졌어요" 표시.

## 5. AI 판정으로 바꿀 때 — 바뀌는 곳은 한 군데

```
// 지금 (MVP, 테스트 입력)
public void OnJudgeConfirmed()
{
    bool[] present = new bool[3];
    for (int i = 0; i < 3; i++)
        present[i] = featureToggles[i].isOn;
    ShowResult(present);
}
```

```
// 나중 (AI 판정)
byte[] png = board.Texture.EncodeToPNG();
// → 프롬프트 + png 를 LLM 에 전송
// ← {"features":[{"featureId":"F1",
//      "status":"present"}, ...]}
present[i] = (status == "present");
ShowResult(present);   // 그대로 재사용
```

> 기획서 6장의 JSON 구조(`inputStatus`, `features[].status`)를 받아 `bool[]` 로만 바꿔주면 결과·재도전·비교 화면은 손대지 않아도 됩니다.

## 6. 폴더 구조

```
FindDokkaebi/
├ Assets/
│  ├ Scripts/               게임 코드 (빌드에 포함)
│  │  ├ GameManager.cs      게임 흐름
│  │  ├ DrawingBoard.cs     그림판
│  │  └ StageData.cs        단계 데이터 양식
│  ├ Editor/                에디터 전용 (빌드에 안 들어감)
│  │  ├ MvpSceneBuilder.cs  씬 자동 조립 (Tools 메뉴)
│  │  └ MvpAutoBuild.cs     Game 씬이 없으면 자동으로 한 번 조립
│  ├ Data/Stage1_Bat.asset  1단계 데이터
│  ├ Sprites/               Bat_Original.png · Bat_Wrong.png (임시 그림)
│  ├ Scenes/Game.unity      게임 씬 (하나뿐)
│  └ Fonts/                 NotoSansKR SDF (한글)
└ Docs/                     MVP_구조_설명.md · 이 PDF
```

## 7. 씬(Hierarchy) 구조

```
Game
├ Main Camera
├ EventSystem                (InputSystemUIInputModule — 마우스·터치를 UI 로)
├ GameManager                ← GameManager.cs
└ Canvas                     (Scale With Screen Size, 1080 x 1920)
   ├ Background
   ├ IntroPanel     Title · Story · StartButton ──────────▶ StartObserve()
   ├ ObservePanel   Guide · TargetImage · Countdown
   ├ DrawPanel      Guide · Board(DrawingBoard)
   │                PenButton ─▶ UsePen()   EraserButton ─▶ UseEraser()
   │                ClearButton ─▶ Clear()  SubmitButton ─▶ Submit()
   ├ JudgePanel     Waiting · TestGuide · FeatureToggle1~3
   │                ConfirmButton ──────────────────────▶ OnJudgeConfirmed()
   ├ ResultPanel    ResultTitle · ReturnedItem · FeatureList
   │                RetryButton ─▶ Retry()   CompareButton ─▶ ShowCompare()
   ├ ComparePanel   MyDrawing · Original · FeatureHint
   │                RestartButton ──────────────────────▶ Restart()
   └ HUD            SnackText · StageText · TimerText   (맨 아래 = 맨 앞에 그려짐)
```

## 8. 확장 로드맵

| 다음 기능 | 고칠 곳 |
|---|---|
| 2~5단계 | StageData 파일 추가 + GameManager 에 `StageData[] stages`, 다음 단계 버튼 |
| 삼원색 펜 | DrawingBoard 에 `SetColor(Color)` + 색 버튼 3개 |
| 선 굵기 / 되돌리기 | DrawingBoard 에 `SetPenSize(int)`, 획마다 `pixels` 복사본 저장 |
| 혼색 팔레트 | 두 색 평균 = `Color.Lerp(a, b, 0.5f)` → SetColor |
| AI 판정 | OnJudgeConfirmed 만 교체 (5번 참고), 오류 시 `snacks++` |
| 진짜 아트 | Sprites 의 PNG 파일만 같은 이름으로 교체 |